Vulnerability Assessment and Penetration Testing (VAPT) Report

Target Application: DVWA (Damn Vulnerable Web Application)

Tested Vulnerabilities: SQL Injection (SQLi) & Cross-Site Scripting (XSS)

Tester: Mahil Joshi

---

1. Introduction

1.1 Purpose of the Report

This report documents the security assessment conducted on the DVWA application to identify and exploit SQL Injection (SQLi) and Cross-Site Scripting (XSS) vulnerabilities. The objective is to demonstrate security flaws and provide remediation recommendations.

1.2 Scope

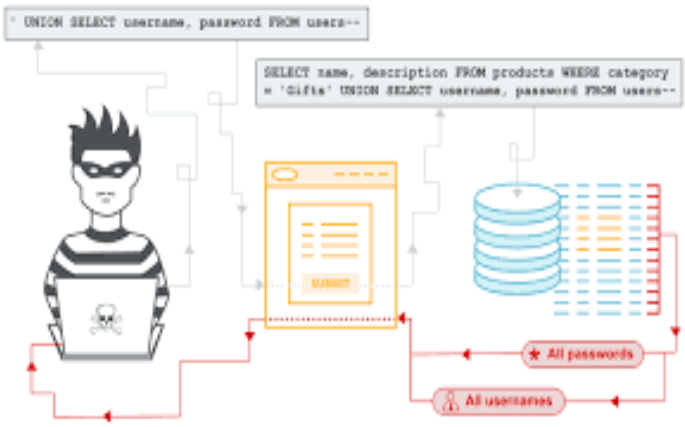
- The assessment was conducted on **DVWA (Damn Vulnerable Web Application)** running in a controlled environment.
- The focus was on **SQL Injection (SQLi) and Cross-Site Scripting (XSS)**.
- Tools Used: **Manual SQL Injection via browser, Payload-based testing, and Direct database enumeration techniques**.

---

2. SQL Injection (SQLi) Assessment

2.1 Vulnerability Overview

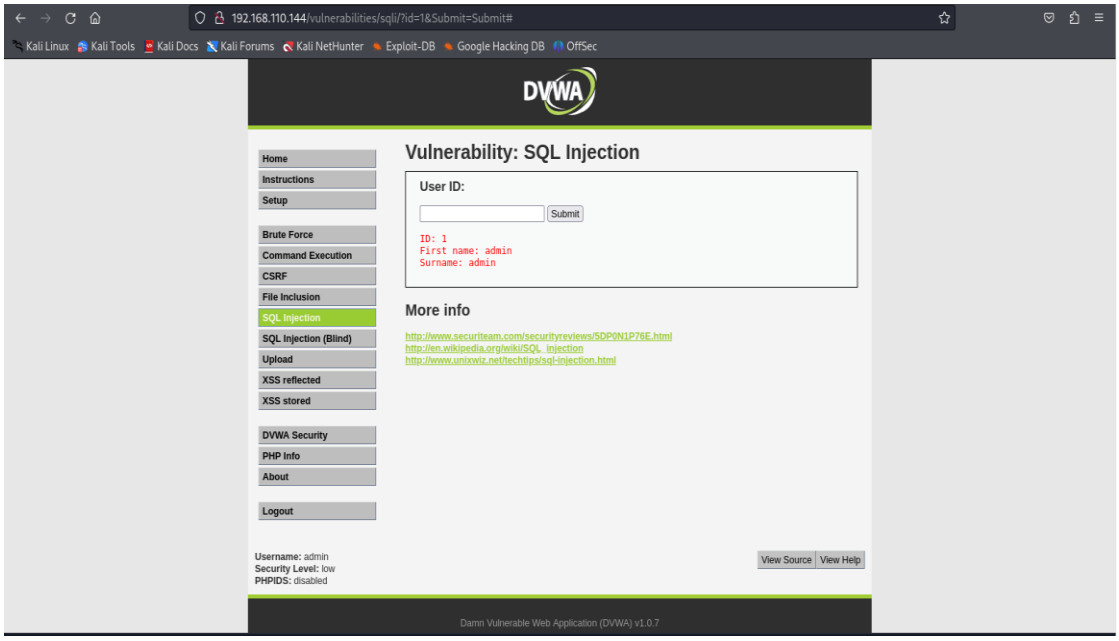
SQL Injection is a web security vulnerability that allows an attacker to interfere with the queries made to a database. This can lead to unauthorized access, data leakage, and database modification.



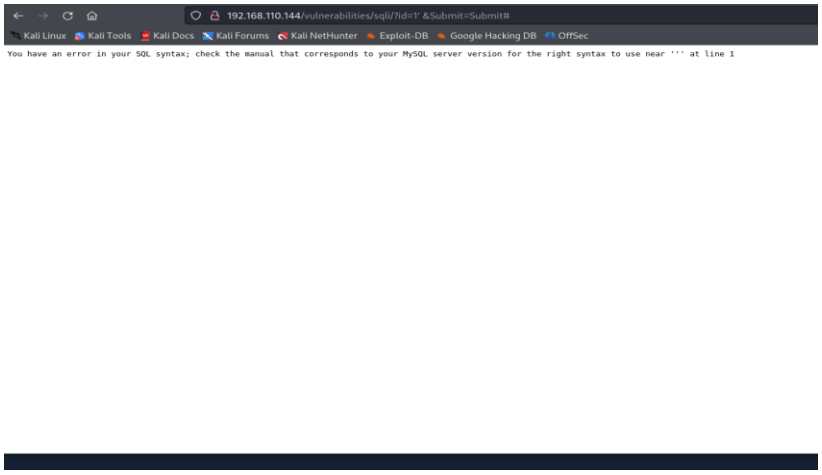
2.2 Testing Methodology

- **Testing Approach:** The testing was performed manually using direct SQL injection payloads in the URL parameters to exploit vulnerabilities in database queries. The primary objective was to extract sensitive information and validate the presence of exploitable SQL Injection vulnerabilities.
- **Techniques Used:**
    1. Error-based SQL Injection
    2. Union-based SQL Injection
    3. Information Schema Enumeration
    4. Data Extraction via SQL Queries
- **Identified Input Fields:** URL parameter id.

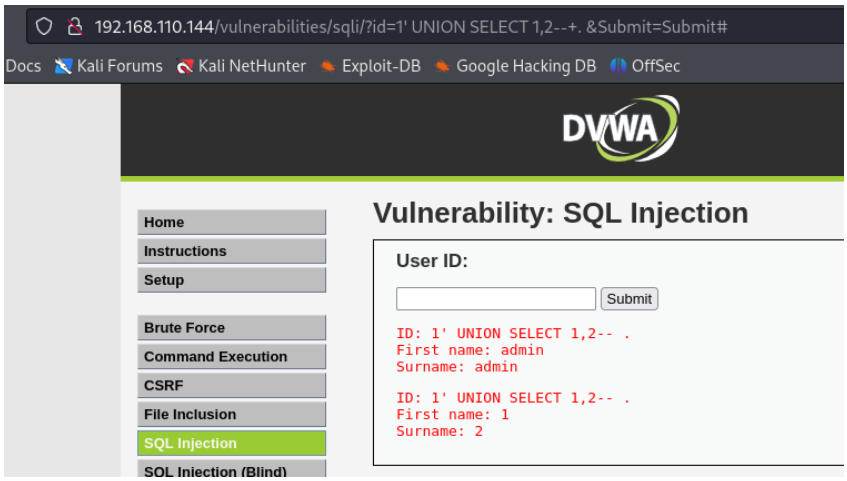
2.3 Proof of Concept (PoC)



1. Inserted a single quote (`'`) in the URL near ?id=1 to check for SQL error.

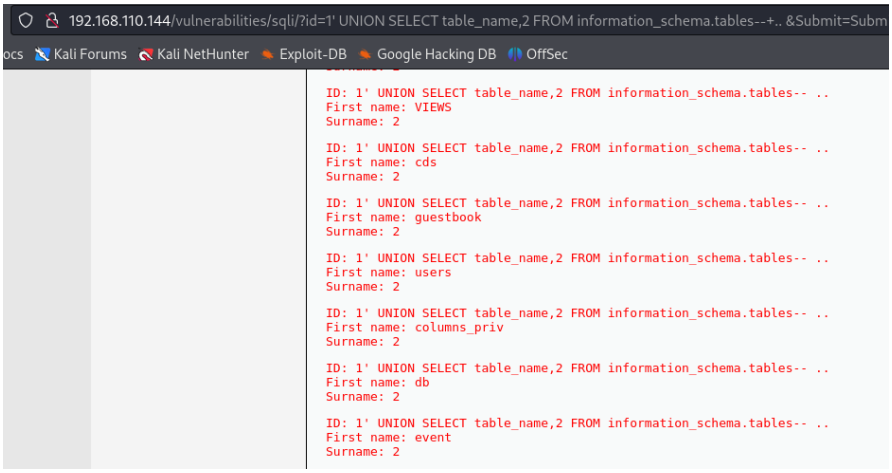


2. Used ORDER BY 1--+ (replacing 1 with 2,3,4,... until the error occurs) to determine the number of columns.
3. Identified the vulnerable columns using UNION SELECT 1,2--+.

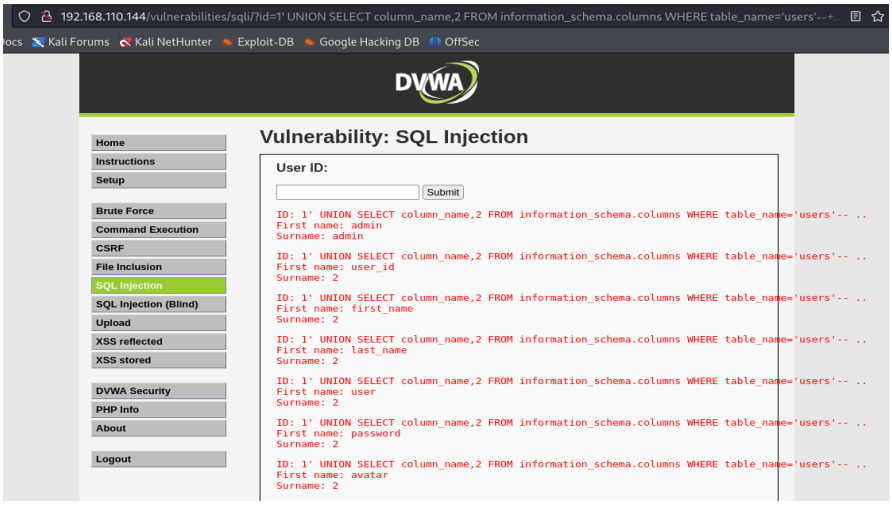


4. Extracted database details with UNION SELECT database(), version()--+.

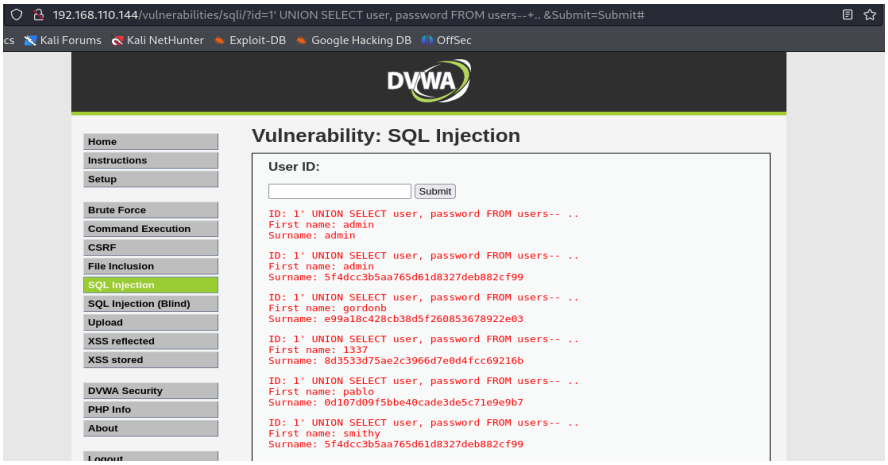
5. Listed available tables using UNION SELECT table_name,2 FROM information_schema.tables--+.



6. Retrieved column names from `users` table using UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'--+.



7. Extracted usernames and password hashes using UNION SELECT user, password FROM users--+.



8. Now we can crack this hashed passwords using hash cracking tools.

9. The entire process was executed via URL-based SQL injection on http://192.168.110.144/vulnerabilities/sqli/?id=1&Submit=Submit#

2.3 Impact

- Unauthorized access to sensitive data.
- Database structure exposure.
- Potential for database deletion or modification.

2.4 Remediation

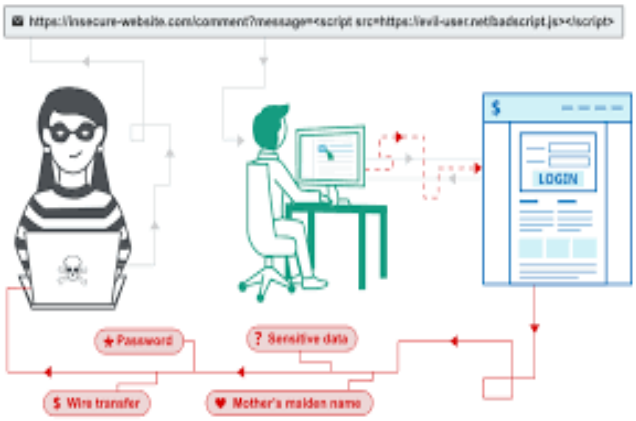
- Use **prepared statements (parameterized queries)** to prevent SQL Injection.
- Implement **input validation and sanitization**.
- Apply **least privilege access** to database users.

---

3. Cross-Site Scripting (XSS) Assessment

3.1 Vulnerability Overview

Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages viewed by users, leading to session hijacking, phishing attacks, and data theft.



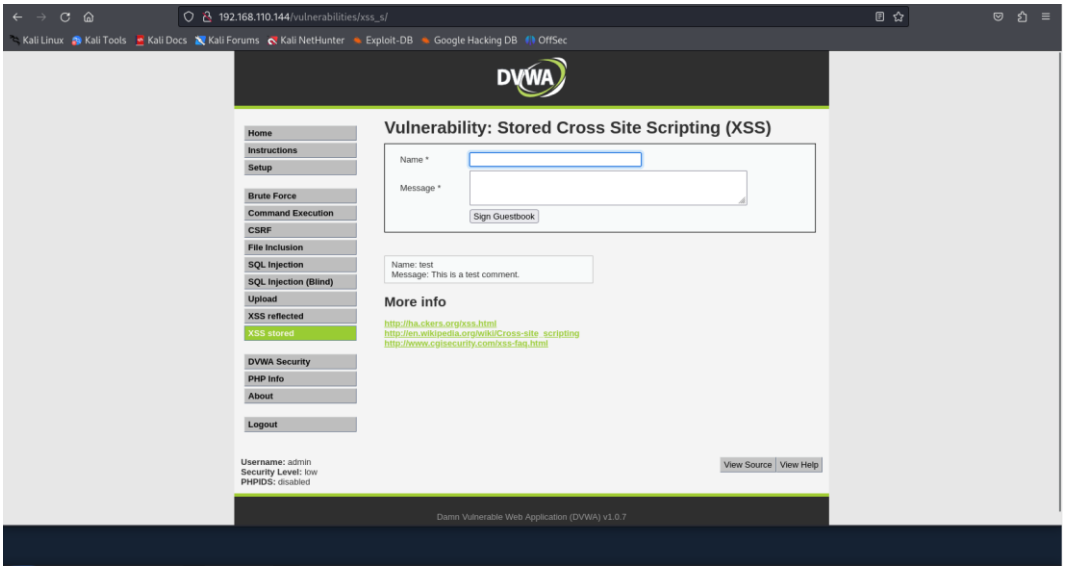
3.2 Testing Methodology

- **Identified Input Fields:** Name sections, Search bars, and Guestbooks.
- **Injected Payloads:**
  - <script>alert('XSS Found');</script>
- **Observations:**
  - Successfully executed JavaScript on the victim's browser.
  - Detected stored and reflected XSS vulnerabilities.

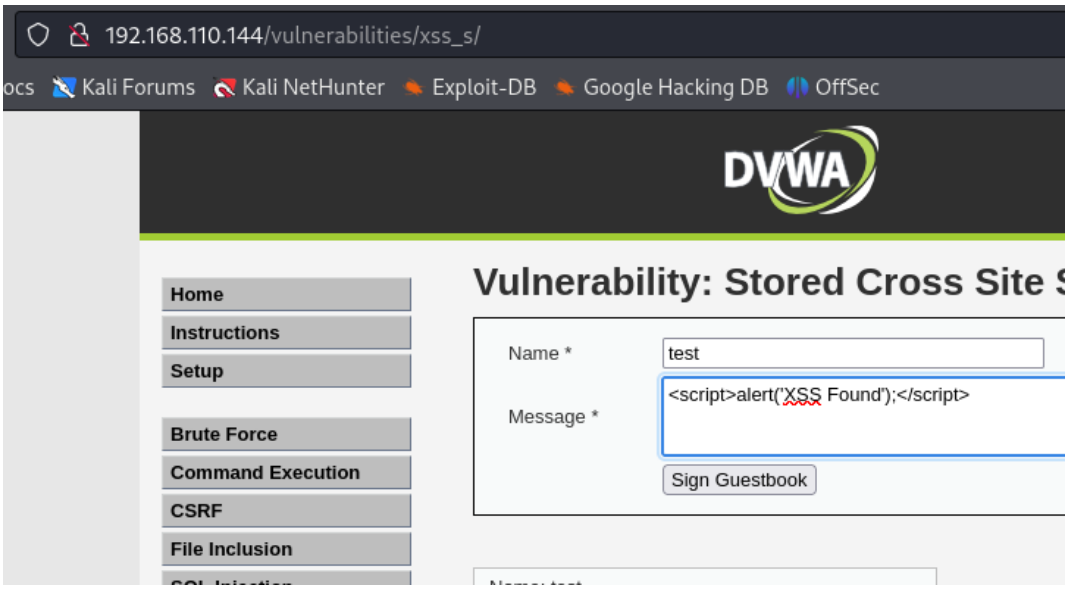
3.3 Proof of Concept (PoC)

Scenario: Stored XSS in Guestbook Section

1. Navigated to the guestbook section.

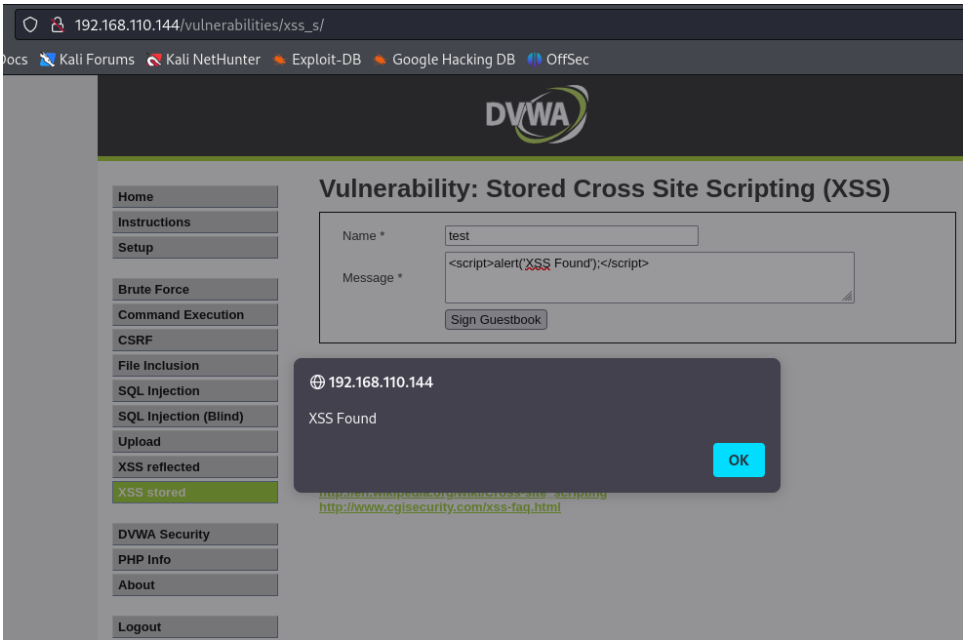


2. Entered the payload : <script>alert('XSS Found');</script> .



3. Submitted the message in guestbook.

4. On reloading the page, the JavaScript executed, displaying the alert.



3.4 Impact

- Execution of malicious scripts on victim browsers.
- Theft of session cookies and user credentials.
- Potential phishing attacks and redirection to malicious sites.

3.5 Remediation

- **Sanitize and escape** user inputs before rendering them in the browser.
- Implement **Content Security Policy (CSP)** to restrict script execution.
- Encode special characters to prevent script execution.

---

4. Conclusion & Recommendations

4.1 Summary of Findings

- **SQL Injection (SQLi) was successfully exploited**, allowing unauthorized access to the database.
- **Cross-Site Scripting (XSS) vulnerabilities were identified and exploited**, demonstrating the risk of malicious script execution.

4.2 Security Recommendations

- Implement **parameterized queries** to mitigate SQLi risks.
- Enforce **input validation and sanitization** across all fields.
- Implement **HTTP-only and Secure flags on cookies** to prevent XSS-based attacks.
- Regularly conduct **security audits and penetration tests**.

---

5. Appendix

5.1 Tools Used

- **Manual SQL Injection via URL parameters** – Exploiting database through direct query execution.
- **Information Schema Enumeration** – Extracting database metadata.
- **Direct Browser-based Payload Testing** – Injecting malicious scripts manually.

5.2 References

- OWASP SQL Injection Guide: https://owasp.org/www-community/attacks/SQL_Injection
- OWASP XSS Prevention Guide: https://owasp.org/www-community/attacks/xss/

[End of DVWA Report]